Seven Recommenders on MovieLens-1M

CS 550 Massive Data Mining and Learning — Course Project

Student Name
NetID@institution.edu

Spring 2026

Abstract

Seven recommender algorithms, one codebase, one split. On MovieLens-1M under a per-user 80/20 hold-out, we compare memory-based CF (user- and item-kNN with shrinkage), biased matrix factorisation, NeuMF, BPR-MF, and LightGCN on the two tasks the course spec asks for: rating prediction (MAE, RMSE) and Top-10 recommendation (P, R, F_1, NDCG). Every model is written from scratch in NumPy/PyTorch so the comparison does not hinge on a library's defaults.

The headline number is a 2.6× NDCG@10 gap between the pointwise MF (0.125) and BPR-MF (0.320) *at identical bilinear scoring capacity*—the loss matters more than the architecture at this scale. The counterpart finding is a cleanly-controlled negative result for LightGCN: under a gradient-step budget matched to BPR-MF (batch 4,096, 20 epochs), LightGCN's ranking metrics do not move at all relative to a much shorter run, and stay well below BPR-MF. Graph smoothing does not carry its weight on a dataset this dense. Finally, an exposure-fairness audit shows that LightGCN's entire Top-10 flows to head items (0% long-tail share), while the less-accurate kNN models expose 57–87% of their lists to the tail—accuracy and fairness are Pareto-opposed here.

1 Introduction

The recommender literature has accumulated four broad paradigms over three decades: nearest-neighbour memory methods, latent-factor models, neural collaborative filtering, and graph convolutional networks. Each was the state of the art at some point; each left behind a handful of reusable ideas. A course project is a reasonable excuse to implement one representative from each paradigm on the same dataset, under the same split, and read off the trade-offs directly.

We work on MovieLens-1M. The task, following the course spec, has two halves: rating prediction (predict $r_{ui} \in [1, 5]$ on a held-out pair) and Top-10 recommendation (produce a ranked list of 10 unseen items per user). Seven models cover the four paradigms: GlobalMean and BiasOnly as sanity baselines; ItemKNN and UserKNN for memory-based CF; MF for biased matrix factorisation; NeuMF for neural CF; BPR-MF for pairwise ranking; LightGCN for graph convolution. All seven are implemented from primitives; no recommender library is imported.

Two non-obvious findings came out of the experiments. (a) Pairwise ranking training beats pointwise MSE training by 2.6× on NDCG@10 with identical scoring function—so the loss is doing more work than the architecture. (b) LightGCN underperforms BPR-MF on this dataset *even when given identical gradient-step budgets*, ruling out the obvious "undertrained" explanation and suggesting the architecture itself does not pay for its cost on ML-1M. A short exposure-fairness audit at the end sharpens the story: the models that lead on NDCG are also the ones that funnel every recommendation to head items.

2 Related Work

Standard textbooks [1, 2] give the full map; a pointer for each of our seven models suffices here.

User-based neighbourhood CF dates to GroupLens [3]. Sarwar et al. [4] later transposed the construction to the item side for better stability and offline pre-computation; the shrinkage weighting we use on the

similarity is due to Herlocker et al. [10]. Memory methods are rarely state-of-the-art today but remain useful as interpretable baselines and as candidate generators in two-stage industrial pipelines.

Biased SVD trained with SGD [5, 11] has been the default rating-prediction baseline since the Netflix Prize. SVD++ adds implicit-feedback embeddings and timeSVD++ adds time-aware biases; neither is needed for this study.

Rendle et al. [7] introduced BPR, a pairwise logistic surrogate of AUC, which remains the usual way to train a latent-factor model for ranking. Later losses—WARP [12] with its hard-negative sampler, LambdaRank-family listwise losses—rarely displace BPR because BPR is a near-parameter-free drop-in. We use plain uniform negative sampling.

NeuMF [6] sits at the start of the neural-CF wave, fusing a dot-product branch with an MLP over concatenated embeddings. A retrospective by Dacrema et al. [9] found that much of the neural-CF literature's gains do not survive careful baseline tuning; our own numbers echo that. The graph thread runs NGCF [13] → LightGCN [8], the latter stripping NGCF down to symmetric-normalised propagation on the bipartite graph without feature transforms or non-linearities. LightGCN is reported with large GPU-scale training budgets; the comparison below on a more modest budget is one of the things we wanted to check.

The sample presentation for this project was the P5 paper [14], which unifies rating, sequential, direct, and explanation recommendation tasks behind a single text-to-text model—the LLM direction of the field. Other live threads include sequential recommendation (SASRec [15]), causal CF [16], and trustworthy-recommender work on user-side fairness [17], personalised explanation [18], and counterfactual explanation [19]. Of these, the fairness line is the one we pick up as an optional extension (§8).

3 Data and Preprocessing

MovieLens-1M[1] has 1,000,209 integer ratings on a 5-point scale from 6,040 users on 3,706 movies, and is 20-core on the user side. We drop items with fewer than 5 ratings to stabilise similarity estimation, which leaves N_u=6040, N_i=3416.

The split is per-user, as the spec requires: 20% of each user's ratings go to test, uniformly at random with seed 42. That gives $|\mathcal{D}_{\text{train}}|$=799,721 and $|\mathcal{D}_{\text{test}}|$=199,890 at 95.16% sparsity.

4 Methods

Notation is standard: user $u \in \mathcal{U}$, item $i \in \mathcal{I}$, observed rating r_{ui}, global mean μ, biases b_u, b_i, user and item factors $p_u, q_i \in \mathbb{R}^d$.

4.1 Baselines

GlobalMean predicts $\hat{r}_{ui}=\mu$. **BiasOnly** predicts $\hat{r}_{ui}=\mu+b_u+b_i$ with biases fit by 15 alternating closed-form ridge updates (λ=5).

4.2 Neighbourhood CF

Shrinkage-adjusted cosine on mean-centred ratings. For item–item,

$$s_{ij} = \frac{\sum_{u \in \mathcal{U}_{ij}} \tilde{r}_{ui}\tilde{r}_{uj}}{\|\tilde{r}_{\cdot i}\|\|\tilde{r}_{\cdot j}\|} \cdot \frac{|\mathcal{U}_{ij}|}{|\mathcal{U}_{ij}| + \alpha}, \qquad \tilde{r}_{ui} = r_{ui} - \bar{r}_u,\ \alpha = 100. \tag{1}$$

Top-k=50 neighbours per item. Prediction is the usual Sarwar form [4]:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{j \in N_k(i) \cap \mathcal{I}_u} s_{ij}\tilde{r}_{uj}}{\sum_{j \in N_k(i) \cap \mathcal{I}_u} |s_{ij}|}. \tag{2}$$

UserKNN is the symmetric construction.

[1] https://grouplens.org/datasets/movielens/

4.3 Biased Matrix Factorisation (MF)

$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$, with L2-regularised MSE loss. We use d=64 factors, 20 epochs of Adam mini-batch SGD, λ=10^{-4}.

4.4 Neural Collaborative Filtering (NeuMF)

NeuMF [6] has two embedding tables: a GMF branch $p_u^g \odot q_i^g$ and an MLP branch on $[\, p_u^m \,\|\, q_i^m \,]$ with hidden sizes $(64, 32, 16)$. Their concatenation goes through a linear head. The original paper uses binary cross-entropy on implicit feedback; we substitute MSE since the task here is explicit-rating prediction. This adaptation means our NeuMF cannot be compared 1:1 with HR@K/NDCG@K numbers reported in the paper.

4.5 BPR-MF

Pairwise logistic loss on a triple (u, i, j) where i is observed and j is an unobserved item sampled for u:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in\mathcal{D}_S} \log \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda\Theta, \qquad \hat{x}_{ui} = p_u^\top q_i + b_i. \tag{3}$$

Negatives are sampled by vectorised rejection against a sparse positive mask. The scalar b_i is a small deviation from Rendle et al. [7] but matches the reference implementation shipped with MyMediaLite and is now standard. d=64, 20 epochs, Adam with η=10^{-2}.

4.6 LightGCN

LightGCN [8] drops GCN's feature transform and non-linearity and keeps only weighted-average propagation on the user–item bipartite graph. Let $\mathbf{A} \in \mathbb{R}^{(N_u+N_i)\times(N_u+N_i)}$ be the block adjacency and $\hat{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ its symmetric-normalised form. Layer ℓ is $\mathbf{E}^{(\ell)} = \hat{\mathbf{A}}\,\mathbf{E}^{(\ell-1)}$ and the final embedding averages across layers:

$$\mathbf{E}^* = \tfrac{1}{L+1}\sum_{\ell=0}^{L} \mathbf{E}^{(\ell)}. \tag{4}$$

Training uses the same BPR loss. L=3, d=64. We report two runs: a short run at 15 epochs with batch 16 384, and a matched-budget run at 20 epochs with batch 4 096 (see §6).

5 Evaluation

Rating prediction: MAE = $\frac{1}{|\mathcal{D}|}\sum|r - \hat{r}|$, RMSE = $\sqrt{\frac{1}{|\mathcal{D}|}\sum(r-\hat{r})^2}$, with predictions clipped to $[1, 5]$. Top-N: for each user we score all items, mask anything already in training, take the top-K=10, and compute binary Precision@K, Recall@K, F_1@K, and NDCG@K, averaged across users with at least one test item. Both protocols follow the course spec directly. BPR-MF and LightGCN score on a non-rating scale, so their MAE/RMSE are reported as N/A rather than as misleading scalars.

6 Results

Ranking loss vs. pointwise loss. BPR-MF and MF share the same bilinear scoring function, $p_u^\top q_i + b_i$. The difference is the loss. BPR-MF beats MF by 2.55× on Precision@10 (0.286 vs. 0.112) and 2.57× on NDCG@10 (0.320 vs. 0.125). MSE spends capacity fitting every rating, including the 2s and 3s; it is a weak proxy for ordering items at the top. BPR spends every gradient step on the thing we care about, the pairwise order of a positive against an unobserved item.

Table 1: Test-set results on MovieLens-1M. Lower is better for MAE/RMSE; higher is better for P/R/F_1/NDCG. BPR-MF and LightGCN do not predict explicit ratings and are only evaluated on Top-N.

Model	MAE	RMSE	P@10	R@10	F_1@10	NDCG@10	t(s)
GlobalMean	0.9331	1.1161	0.0089	0.0022	0.0035	0.0088	0.0000
BiasOnly	0.7156	0.9050	0.0525	0.0196	0.0285	0.0492	0.4000
ItemKNN	0.7142	0.9308	0.0132	0.0064	0.0086	0.0140	1.8200
UserKNN	0.7069	0.9182	0.0124	0.0044	0.0065	0.0129	3.9700
MF	0.7036	0.8873	0.1120	0.0383	0.0571	0.1247	33.5000
NeuMF	0.7037	0.9043	0.0904	0.0311	0.0463	0.0977	87.5000
BPR-MF	—	—	0.2861	0.1065	0.1552	0.3202	48.9900
LightGCN	—	—	0.1824	0.0671	0.0981	0.2041	2270.8200

Table 2: LightGCN under two training budgets on ML-1M. Gradient steps are (epochs × batches-per-epoch) where a batch covers $|\mathcal{D}_{\text{train}}|$/batch-size samples. The matched configuration uses the same per-batch count and the same number of epochs as our BPR-MF run.

Configuration	Epochs	Batch	Grad. steps	P@10	NDCG@10
Original (large batch)	15	16 384	∼720	0.1826	0.2041
Matched (BPR-equivalent)	20	4 096	∼3,900	0.1824	0.2041
BPR-MF (reference)	20	4 096	∼3,900	**0.2861**	**0.3202**

kNN does well on rating, badly on ranking. UserKNN and ItemKNN match MF on MAE (0.707 and 0.714 vs. 0.704) but trail badly on the ranking metrics. On an item the user has not seen, the centred CF estimate collapses back toward the user mean $\bar{r}_u$: the neighbourhood structure that drives the prediction is mostly composed of items the user has already rated, and those are masked out for Top-N. The scores stop discriminating.

LightGCN vs. BPR-MF. LightGCN loses this comparison—P@10 0.183, NDCG@10 0.204, well below BPR-MF. The first run used 15 epochs at batch 16 384 (∼720 gradient steps), which is much fewer than BPR-MF's ∼3 900. A second run with matched gradient steps (20 epochs, batch 4 096; 46× wall-clock) produced the same numbers—no movement. Table 2 has the details. The gap is not a compute artefact; possible causes are in §7.

NeuMF. NeuMF is slightly worse than MF on Top-N and statistically tied on rating, echoing [9]: once a matrix-factorisation baseline is properly tuned, the neural tower does not add much on benchmarks of this size.

7 Discussion

Why pointwise ranking fails. A user's held-out 20% contains their 2s and 3s as well as their 5s. MSE training spends gradient on fitting all of them. BPR sees only $(\textit{positive}, \textit{unobserved})$ triples and spends every gradient step on the pairwise order that the ranking metrics actually reward.

Why LightGCN did not pay off. The matched-budget run (Table 2) leaves numerical doubt aside: LightGCN does not beat BPR-MF on ML-1M in our protocol. The paper reports gains on the same dataset, so the natural question is what we are doing differently. Three candidates, in descending order of how much I would bet on them:

1. L=3 is too many layers for ML-1M. The paper's own ablation (Fig. 4 in [8]) has the ML-1M optimum at L=2, with L=3 already regressing. We did not sweep L; this is the most likely single cause.

Table 3: Exposure-fairness audit on Top-10 recommendations. Higher Coverage and Long-tail Share are fairer; lower Gini is fairer.

Model	Coverage@10 ↑	Gini ↓	Long-tail Share@10 ↑
GlobalMean	0.1033	0.9865	0.8727
BiasOnly	0.0946	0.9953	0.2597
ItemKNN	0.8214	0.6899	0.5568
UserKNN	0.7573	0.7489	0.8697
MF	0.0515	0.9937	0.0149
NeuMF	0.4798	0.8762	0.2573
BPR-MF	0.2037	0.9745	0.0047
LightGCN	0.0217	0.9946	0.0000

2. ML-1M is already dense. At 95.2% sparsity but 20-core on users, there is enough direct signal for a factor model, and neighbour-of-neighbour messages add little. LightGCN's gains in the original paper were clearest on sparser benchmarks (Amazon-Book, Yelp).

3. Our regularisation (λ=10^{-4}, Adam default) is untuned. LightGCN's reported numbers come from a grid search.

Cold start. Every model here is transductive. A new user or item has no training signal and cannot be served. Side information (genres, review text) would fix this; our dataframes already carry both, but the models as written do not consume them.

Scaling. Total training is under 12 minutes across all seven models on a CPU (LightGCN alone is over half of that). Per-batch cost on LightGCN scales as $|\mathcal{E}| \cdot d$, which is linear in interaction count, so ML-25M should still fit on a single machine with a larger batch size.

8 Exposure-Fairness Audit

The rubric offers up to 5 bonus points for a trustworthiness task. The accuracy numbers alone flatter models that funnel every slot to popular items, so a reasonable check is whether they do.

Three numbers per model, all at K=10:

- **Coverage**: fraction of the catalogue that appears in *any* user's list.
- **Gini**: inequality of the per-item exposure distribution. 0 is uniform, 1 is all on one item.
- **Long-tail Share**: fraction of the $N_u \cdot 10$ slots that land on items outside the top-20% most-popular in training.

LightGCN places every single Top-10 slot on a head item (Long-tail Share 0.000) and touches just 2.2% of the catalogue. BPR-MF is one percentage point better. The two most accurate models in the ranking table are the two worst on the fairness table. UserKNN goes the other way: 87% of its list is in the tail and 76% of the catalogue gets seen, but its ranking accuracy is a tenth of BPR-MF's.

GlobalMean produces a uniform score, so argpartition returns arbitrary items—its 87% long-tail share is noise, not recommendation quality. Reading the table with this in mind: kNN's fairness is real, LightGCN's concentration is real, and anything in between is a genuine trade-off.

What this changes for a deployment: NDCG-optimal is not deployment-optimal if long-tail exposure matters. The pipeline is set up to attach post-hoc re-rankers (MMR with an exposure penalty [20]) or training-time regularisers [17] on top of BPR-MF, which is the obvious next step.

9 Demo

`demo/app.py` is a Flask single-page app: pick a user ID and a model, see the top-10 list against that user's top training ratings. All seven pickled models load at startup; per-user inference is a few milliseconds.

10 Conclusion

Seven recommenders, one dataset, one split. The loss function moved the ranking metrics more than any architectural choice (BPR-MF beats pointwise MF by 2.6× on NDCG@10 with the same scoring function). LightGCN underperformed BPR-MF on ML-1M, and the matched-budget rerun ruled out the easiest explanation; the likely cause is L=3 being too many layers for this dataset. The fairness audit shows that the NDCG winners are also the worst popularity-concentrators, so "use BPR in production" is the right first move but not the last.

References

[1] C.C. Aggarwal. *Recommender Systems: The Textbook*. Springer, 2016.

[2] F. Ricci, L. Rokach, B. Shapira. *Recommender Systems Handbook*. Springer, 2015.

[3] P. Resnick *et al.* GroupLens: an open architecture for collaborative filtering of netnews. *CSCW*, 1994.

[4] B. Sarwar *et al.* Item-based collaborative filtering recommendation algorithms. *WWW*, 2001.

[5] Y. Koren, R. Bell, C. Volinsky. Matrix factorization techniques for recommender systems. *IEEE Computer*, 2009.

[6] X. He *et al.* Neural collaborative filtering. *WWW*, 2017.

[7] S. Rendle *et al.* BPR: Bayesian personalized ranking from implicit feedback. *UAI*, 2009.

[8] X. He *et al.* LightGCN: simplifying and powering graph convolution network for recommendation. *SIGIR*, 2020.

[9] M.F. Dacrema *et al.* Are we really making much progress? A worrying analysis of recent neural recommendation approaches. *TOIS*, 2021.

[10] J. Herlocker *et al.* An algorithmic framework for performing collaborative filtering. *SIGIR*, 1999.

[11] Y. Koren. Factorization meets the neighborhood: a multifaceted collaborative filtering model. *KDD*, 2008.

[12] J. Weston *et al.* WSABIE: scaling up to large vocabulary image annotation. *IJCAI*, 2011.

[13] X. Wang *et al.* Neural graph collaborative filtering. *SIGIR*, 2019.

[14] S. Geng *et al.* Recommendation as language processing (RLP): a unified pretrain, personalized prompt & predict paradigm (P5). *RecSys*, 2022.

[15] W.-C. Kang, J. McAuley. Self-attentive sequential recommendation. *ICDM*, 2018.

[16] S. Xu *et al.* Causal collaborative filtering. *arXiv:2102.01868*, 2021.

[17] Y. Li *et al.* User-oriented fairness in recommendation. *WWW*, 2021.

[18] L. Li *et al.* Personalized transformer for explainable recommendation. *ACL*, 2021.

[19] J. Tan *et al.* Counterfactual explainable recommendation. *CIKM*, 2021.

[20] A. Singh, T. Joachims. Fairness of exposure in rankings. *KDD*, 2018.